Lab Assignment 1: Task Model Inference and Timing Behaviour Validation

Real-Time Computing and Embedded Systems

Overview

In this lab, you will implement a small Python program that analyzes an execution trace (available in `trace.txt`) collected from an embedded system. From the trace, you will:

(a) infer tasks, jobs, their parameters, and

(b) validate periodic timing behaviour

Deadline: The assignment deadline is on **29/12/2026 4 PM**.
Note: Source code `solution.py` with initial parsing and printing implementation is provided.

Input Format

Each line of the input file is a single event

$$\langle \texttt{TASK_ID} \rangle \ \langle \texttt{EVENT} \rangle : \ \langle \texttt{TIMESTAMP} \rangle$$

where:

- `TASK_ID` is an identifier without spaces (e.g., `A`, `B`),
- `EVENT` $\in \{\texttt{START}, \texttt{FINISH}\}$, and
- `TIMESTAMP` is a positive integer. The file does *not* report time units, but you should treat these values as *miliseconds* (ms).

First 7 events from `trace.txt`:
B FINISH: 988000
A START: 990000
B START: 990000
A FINISH: 991000
B FINISH: 993000
A START: 995000
B START: 995000
...

First, implement support for *ignoring* the unmatched first occurrence of a `FINISH` with no corresponding (preceding) `START`, and the unmatched last `START` with no corresponding `FINISH` (e.g., not observed due to a trace stop). It may simply be that your tracing starts (or ends) at a specific time (not necessarily system startup/end), and we want to analyze only **completely observed jobs** for which the start and finish times are known.

[*points: this functionality is a prerequisite for getting points from the following exercises*]

Definitions

Let $\Gamma = \{\tau_A, \tau_B, ...\}$ be the set of tasks observed in the file. The j-th completely observed *job* of task $\tau_X \in \Gamma$ is characterized by a matched pair of events (`START`, `FINISH`) representing the j-th start time $s_{X,j}$ and j-th finish time $f_{X,j}$.

Exercises

Exercise 1 (Basic Inference) [5 points]. Implement a function reporting the following:

- identified tasks [1 points],
- identified number of complete jobs for each task [2 points],
- increasingly time-ordered events from all completely observed jobs [2 points].

Exercise 2 (Job Inference) [5 points] Implement a function that *for each task* $\tau_X \in \Gamma$, lists its jobs with corresponding parameters:

- Job index, starting from 1 to the number of completeely observed jobs of τ_X,
- Start time, and
- Finish time.

Present the results in a readable printout (grouped by task) with e.g.,
"*Task A: j=1, s=990000, f=991000,...*" as the printing format.

Exercise 3 (Periodic Model Check) [10 points]. For each task τ_X, a user can supply two integers: offset Φ_X and period $T_X > 0$, assumed to be milliseconds.

Deadlines and arrivals. Let us define the absolute deadline and release for job $\tau_{X,j}$ as

$$d_{X,j} \;=\; \Phi_X + j \cdot T_X, \qquad r_{X,j} \;=\; \Phi_X + (j-1) \cdot T_X.$$

Job $\tau_{X,j}$ **meets** its deadline iff $f_{X,j} \le d_{X,j}$. Task τ_X is **compliant** with the periodic model $\langle \Phi_X, T_X \rangle$ iff for every completely observed job $\tau_{X,j}$, $r_{X,j} \le s_{X,j} \quad \text{and} \quad f_{X,j} \le d_{X,j}$.

Implement a function that

(a) For each task, lists the jobs that *violate* their respective deadlines.
Print j, $r_{X,j}$ $s_{X,j}$, $f_{X,j}$, $d_{X,j}$ as follows "*j=..., r =... s=..., f=..., d =...*" [5 points]

(b) For each task, reports whether it is compliant with the given periodic model, and if not, the first violating job information (full print as above). [5 points]

All exercises are considered complete if they are working on the example trace. In Exercise 3, it is mandatory to report one compliant and one non-compliant periodic model.

Deliverables & Usage

Submit:

- your updated `solution.py` Python source file, **with comments**.
- a brief `README.md` describing the implemented functions and how to run the code.